CURSO: SISTEMAS DE INFORMAÇÃO

Programação Orientada a Objetos II

Prof. Yassushi Okada

Logging

Em aplicações reais, erros não podem depender apenas de mensagens no console. É necessário registrar informações de forma persistente para análise posterior. Esse processo é chamado de logging.

Logging permite:

- rastrear erros em produção
- entender o comportamento do sistema
- auditar ações
- facilitar manutenção e debugging

Diferença entre log e exceção

Muitos alunos confundem isso.

Exceção

- representa um erro no código
- interrompe o fluxo
- carrega stack trace

Log

- é um registro de evento
- não necessariamente é erro
- pode representar comportamento normal

Exemplo:

```
logger.Info("Usuário iniciou login"); // log

throw new Exception("Senha inválida"); // exceção
```

Tipos de logs

Logs de negócio

Descrevem ações do sistema:

```
logger.Info("Pedido criado com ID 123");
```

Logs técnicos

Relacionados ao funcionamento interno:

```
logger.Debug("Conexão aberta com banco");
```

Logs de erro

Relacionados a falhas:

```
logger.Error(ex, "Erro ao salvar pedido");
```

Logs de auditoria

Importantes para rastreabilidade:

```
logger.Info("Usuário 10 alterou senha");
```

O problema do excesso de logs

Um erro comum é achar que “quanto mais log, melhor”.

Na prática:

- logs em excesso dificultam análise
- aumentam custo (armazenamento)
- prejudicam performance

Exemplo ruim

```
logger.Debug("Entrou no método");
logger.Debug("Executando linha X");
logger.Debug("Executando linha Y");
```

Isso não agrega valor real.

Logging de qualidade

Um bom log responde:

- o que aconteceu
- onde aconteceu
- com quais dados
- qual foi o impacto

Exemplo bom

```
logger.Error(ex, "Erro ao transferir valor {0} da conta {1} para {2}", valor, origemId,
destinoId);
```

Contexto no logging

Logs sem contexto são quase inúteis.

Sem contexto

```
logger.Error("Erro ao salvar");
```

Com contexto

```
logger.Error(ex, "Erro ao salvar usuário {0}", usuarioId);
```

Correlação de logs

Em sistemas maiores, uma única operação pode gerar vários logs.

Exemplo:

- API recebe requisição
- chama serviço
- chama banco

Sem correlação, você não sabe quais logs pertencem à mesma operação.

Solução

Usar um identificador:

```
var requestId = Guid.NewGuid();

logger.Info("Início da requisição {0}", requestId);
```

Todos os logs daquela operação usam o mesmo ID.

Estrutura do log

Um log bem estruturado normalmente contém:

- timestamp
- nível
- mensagem

- contexto (parâmetros)
- exceção (se houver)

Exemplo típico:

```
2026-05-04 10:00:00 | ERROR | Erro ao salvar usuário 10 | System.Exception: ...
```

Logging estruturado vs texto simples

Texto simples

```
logger.Info("Usuário 10 realizou saque de 200");
```

Problema:

- difícil de filtrar automaticamente

Logging estruturado

```
logger.Info("Usuário {UsuarioId} realizou saque de {Valor}", usuarioId, valor);
```

Vantagem:

- permite consultas
- melhor análise em ferramentas

Performance e logging

Logging impacta performance, principalmente:

- escrita em disco
- logs em excesso
- logs síncronos

Boas práticas:

- usar níveis adequados
- evitar logs desnecessários
- usar buffers (quando possível)

Logging em produção vs desenvolvimento

Desenvolvimento

- mais logs (Debug, Trace)
- foco em diagnóstico

Produção

- menos logs detalhados
- foco em estabilidade
- logs mais relevantes (Info, Warn, Error)

NLog

O NLog é uma biblioteca de logging para .NET que permite:

- registrar logs em diferentes destinos (arquivo, console, banco, etc.)
- configurar níveis de log
- estruturar mensagens
- separar configuração do código

Níveis de log

O NLog trabalha com níveis que representam a gravidade do evento:

Nível	Uso
Trace	Detalhes muito finos (debug profundo)
Debug	Informações para desenvolvimento
Info	Eventos normais da aplicação
Warn	Situações inesperadas, mas não críticas
Error	Erros que afetam funcionalidades
Fatal	Falhas críticas que encerram o sistema

Instalação

Via NuGet:

```
Install-Package NLog
Install-Package NLog.Config
Install-Package NLog.Schema
```

O pacote NLog.Config já cria automaticamente o arquivo de configuração NLog.config.

Estrutura do NLog.config

Exemplo básico:

```
<?xml version="1.0" encoding="utf-8" ?>
<nlog xmlns="http://www.nlog-project.org/schemas/NLog.xsd"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">

  <targets>
    <target name="arquivo" xsi:type="File" fileName="log.txt"
            layout="${longdate} | ${level} | ${message} ${exception}" />

    <target name="console" xsi:type="Console"
            layout="${longdate} | ${level} | ${message}" />
  </targets>

  <rules>
    <logger name="*" minlevel="Info" writeTo="arquivo,console" />
  </rules>

</nlog>
```

Explicação da configuração

Targets (destinos)

Define para onde o log será enviado:

- arquivo (File)
- console (Console)

Layout

Define o formato da mensagem:

- ${longdate} → data completa
- ${level} → nível do log
- ${message} → mensagem
- ${exception} → exceção (se houver)

Rules (regras)

Define:

- quais logs serão capturados
- nível mínimo
- destino

Uso básico no código

Criando o logger

```
using NLog;

class Program
{
    private static readonly Logger logger =
        LogManager.GetCurrentClassLogger();

    static void Main()
    {
        logger.Info("Aplicação iniciada");
    }
}
```

Exemplo 1 — Logs simples

```
logger.Trace("Detalhe técnico");
logger.Debug("Valor de variável X");
logger.Info("Usuário logado");
logger.Warn("Tentativa inválida");
logger.Error("Erro ao salvar dados");
logger.Fatal("Falha crítica no sistema");
```

Exemplo 2 — Logging de exceção

```
try
{
    int x = int.Parse("abc");
}
catch (Exception ex)
{
    logger.Error(ex, "Erro ao converter valor");
}
```

Isso registra:

- mensagem
- stack trace
- tipo da exceção

Exemplo 3 — Logging em aplicação de conta

Classe Conta (Domain)

```
public class Conta
{
    public decimal Saldo { get; private set; }

    public Conta(decimal saldo)
    {
        Saldo = saldo;
    }

    public void Sacar(decimal valor)
    {
        if (valor <= 0)
            throw new ArgumentException("Valor inválido");

        if (Saldo < valor)
            throw new Exception("Saldo insuficiente");

        Saldo -= valor;
    }
}
```

Uso com logging

```
using NLog;

class Program
{
    private static readonly Logger logger =
        LogManager.GetCurrentClassLogger();

    static void Main()
    {
        var conta = new Conta(100);

        try
        {
            logger.Info("Tentando sacar 200");
            conta.Sacar(200);
        }
        catch (Exception ex)
        {
            logger.Error(ex, "Erro ao realizar saque");
        }
    }
}
```

Exemplo 4 — Logging informativo (rastreamento)

```
logger.Info("Iniciando transferência");
logger.Debug("Conta origem: 1");
logger.Debug("Conta destino: 2");
logger.Info("Transferência concluída");
```

Logging com parâmetros

```
int usuarioId = 10;
decimal valor = 200;

logger.Info("Usuário {0} realizou saque de {1}", usuarioId, valor);
```

Boas práticas de logging

Registrar contexto

Evite:

```
logger.Error("Erro");
```

Prefira:

```
logger.Error(ex, "Erro ao salvar pedido do usuário {0}", usuarioId);
```

Não logar informação desnecessária

Evite excesso de logs:

- pode poluir
- dificulta análise
- impacta performance

Usar níveis corretamente

- Debug → desenvolvimento
- Info → fluxo normal
- Error → falhas reais

Nunca esconder exceções

Errado:

```
catch
{
}
```

Correto:

```
catch (Exception ex)
{
    logger.Error(ex, "Erro inesperado");
    throw;
}
```

Separar log de regra de negócio

Evite log dentro do domínio:

```
// evitar dentro da entidade
logger.Info("Sacando valor");
```

Prefira:

- log na camada de serviço ou UI

Exercícios

Exercício 1

Configure o NLog para:

- salvar logs em arquivo
- mostrar no console

Exercício 2

Crie logs para:

- início do programa
- fim do programa

Exercício 3

Implemente:

- conversão de string para inteiro
- log de erro ao falhar

Exercício 4

Crie uma classe Conta e:

- logue tentativa de saque
- logue erro de saldo insuficiente

Exercício 5

Diferencie logs:

- Info → operação normal
- Warn → tentativa inválida
- Error → exceção

Exercício 6

Implemente um fluxo:

- depósito
- saque
- transferência

Registre:

- início
- sucesso
- erro

Exercício 7

Altere o layout do log para incluir:

- data
- nível
- mensagem
- exceção
